DAY 07

Strings & String Methods

INTERMEDIATE PHASE

Session Length	Difficulty	Prerequisites	Day in Course
2 hours	Beginner-Intermediate	Days 02, 06	7 of 30

Instructor Session Guide — 30-Day Python Programming Course

Session Overview

Goal: Equip students to format text cleanly and manipulate strings with confidence — covering formatting styles, the most-used string methods, and an introduction to pattern matching with the `re` module.

Outcome: By the end of this session, students will be able to build formatted, India-contextualized output (receipts, report cards, invoices), clean and transform raw text data, and extract structured information (phone numbers, emails, PIN codes) using regular expressions.

Time	Segment	Focus
0:00 – 0:10	Warm-up & Recap	Review Day 06 data structures
0:10 – 0:35	String Formatting	%, .format(), f-strings, specifiers
0:35 – 1:05	String Methods	split, join, strip, replace, more
1:05 – 1:15	Break	—
1:15 – 1:45	re Module Basics	search, match, findall
1:45 – 2:00	Practice & Wrap-up	Guided examples + Q&A

TIP

Keep the pace brisk through formatting and methods — they are individually simple. Spend the saved time on the re module, since it is the first genuinely new mental model students encounter today.

Prerequisites & Preparation

Requirements

Requirement	Details
Completed Days	Day 02 (Variables), Day 06 (Data Structures)
Python Version	3.12 (from Day 01 setup)
Editor	VS Code with Python extension
Files Needed	day07_01/02/03 concept files, practice problem files

Instructor Pre-Session Checklist

- ■ Confirm all students can import the `re` module (built-in, no install needed)
- ■ Pre-load the three concept files and three practice files on the shared drive
- ■ Prepare 2-3 real-world text snippets to demo regex on-the-fly
- ■ Review the Day 06 recap examples so the warm-up segment is ready

WATCH OUT

The re module ships with Python's standard library — there is nothing to install. Double-check students are on Python 3.10+ if you plan to demo the match statement from Day 03 as a refresher.

Step-by-Step Instruction Guide

STEP 1 — String Formatting (25 min)

What to Say:

"So far, when we've printed variables together, we've used the + operator to glue strings and numbers. That gets messy fast — especially when numbers need rounding or alignment. Today we learn three ways to format text: the old % operator, .format(), and f-strings — which is what you'll use in almost all your code going forward."

```
# f-strings are the modern standard
employee = "Priya Sharma"
salary = 65000
print(f"{employee} earns Rs. {salary} per month.")
# Output: Priya Sharma earns Rs. 65000 per month.

amount = 125000.789
print(f"Rs. {amount:,.2f}")
# Output: Rs. 125,000.79
```

Common Error	Cause	Fix
Curly braces show up literally	Forgot the f prefix	Add f before the opening quote
KeyError in .format()	Named placeholder misspelled	Match placeholder name exactly
ValueError: unknown format code	Invalid specifier (e.g. :,.2x)	Use valid specifiers like :.2f, :,

DONE WHEN

Students can run day07_01_string_formatting.py end-to-end and see every output printed — including the aligned invoice example. That's the signal they're ready to move on.

STEP 2 — String Methods (30 min)

What to Say:

"Now that we can format output, let's talk about cleaning and transforming input. Real data — from forms, files, or APIs — is messy: extra spaces, inconsistent casing, unwanted characters. split(), join(), strip(), and replace() are the tools you'll reach for constantly."

```
raw_phone = "  +91-98765 43210  "
clean = raw_phone.strip().replace("-", "").replace(" ", "")
print(clean)
# Output: +919876543210

fields = "Priya,28,Mumbai".split(",")
print(fields)
# Output: ['Priya', '28', 'Mumbai']
```

Common Error	Cause	Fix
join() raises TypeError	List contains non-string items	Convert with str() before joining
strip() doesn't remove chars	strip() only trims from ends	Use replace() for mid-string chars
split() gives extra empty strings	Multiple consecutive delimiters	Use split() with no args for whitespace

DONE WHEN

Done when students can clean the messy CSV record example themselves — stripping whitespace from every split field and printing each one on its own line.

STEP 3 — re Module Basics (30 min)

What to Say:

"Last piece for today: what if we need to find PATTERNS instead of exact text — like 'any 10-digit number' or 'any email address'? That's what regular expressions are for. We'll use three functions: search(), match(), and findall(). Don't worry about memorizing every symbol today — focus on search vs. match vs. findall, and the basics: \d, \w, \s, and +."

```
import re

text = "Paid Rs.500, Rs.1200, and Rs.75"
amounts = re.findall(r"\d+", text)
print(amounts)
# Output: ['500', '1200', '75']

match = re.match(r"MUM", "MUM2026-0093")
print(match.group() if match else "No match")
# Output: MUM
```

Platform	Command to Run a File
macOS / Linux	python3 day07_03_regex_basics.py
Windows	python day07_03_regex_basics.py

Common Error	Cause	Fix
match() returns None unexpectedly	Pattern isn't at string start	Use search() instead of match()
AttributeError: 'NoneType'	Called .group() on a failed match	Check `if match:` before .group()
Pattern matches too much/little	Missing word boundary \b or +/*	Add \b or adjust quantifiers

DONE WHEN

Done when students can extract all 10-digit phone numbers from a block of sample text using re.findall() with a \b\d{10}\b pattern.

Session Wrap-Up

Student Checklist

- ■ I can format strings using f-strings and format specifiers (:.2f, :,)
- ■ I can split a string into a list and join a list back into a string
- ■ I can clean text using strip() and replace()
- ■ I can use re.search(), re.match(), and re.findall() to find patterns
- ■ I completed all three concept files and attempted the practice set

Key Concepts Recap

Concept	One-Line Summary
f-strings	f"{var:.2f}" — embed expressions with format specifiers
split() / join()	String ↔ list conversions using a delimiter
strip() / replace()	Clean whitespace and substitute substrings
re.search/match/findall	Find patterns anywhere / at start / all matches

Next Day Preview

Day 08 — File I/O: Students will learn to read, write, and append files using open() and the with statement, plus the csv and json modules for structured data — building directly on today's string-cleaning skills.

WATCH OUT

If students are shaky on split()/join(), pair them up for the first 10 minutes of Day 08 before introducing file reading — CSV parsing leans heavily on both.

Quick Reference Card

Daily Workflow Commands

Command	Purpose
`python day07_01_string_formatting.py`	Run concept file 1
`python day07_02_string_methods.py`	Run concept file 2
`python day07_03_regex_basics.py`	Run concept file 3
`python day07_practise_problems_01_string_formatting.py`	Run practice set 1

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Open integrated terminal
F5	Run current file with debugger
Ctrl/Cmd + /	Toggle line comment
Shift + Alt + F	Format document

Tool Summary

Tool / Function	Use Case
`f"{x:.2f}", f"{x:,}"`	Precision and thousands-separator formatting
`str.split(sep)`	Break a string into a list
`sep.join(list)`	Combine a list into a string
`str.strip() / replace()`	Clean whitespace / substitute text
`re.search / match / findall`	Pattern matching and extraction